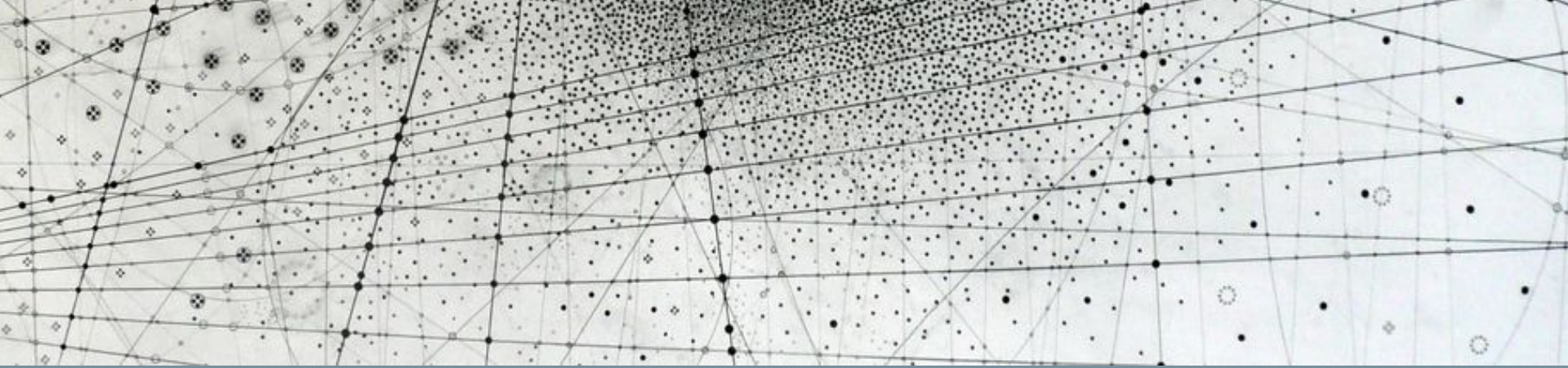




Research Methods in Machine Learning & Physics

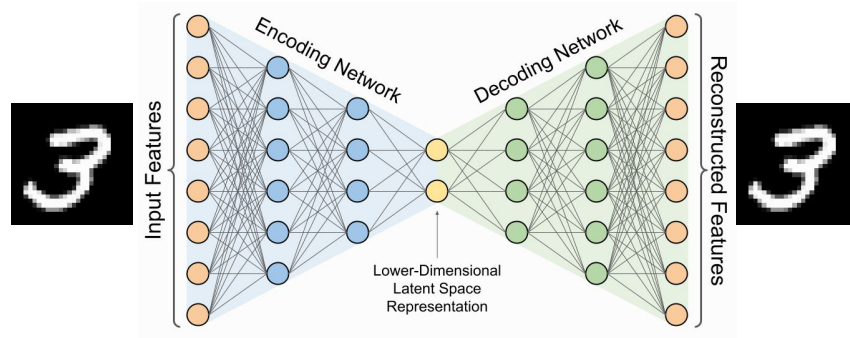
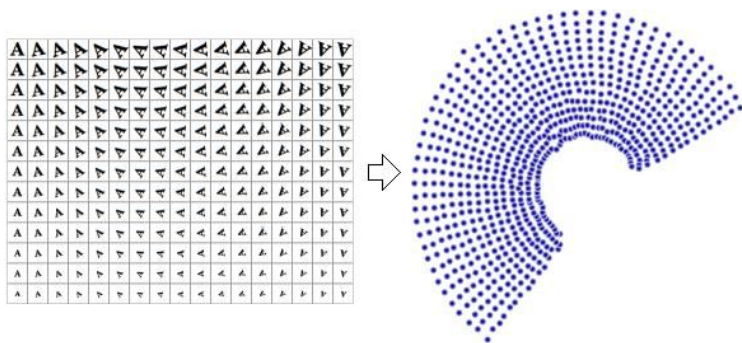
Some course logistics

- **Project #1 is posted and due by class time next Monday** (4pm on 10/6)
 - Upload a .zip file of your notebook (.ipynb & .html formats) + PDF of a 1-2pg. extended abstract
 - Will figure out how to put this in
- **The ML glossary in Overleaf**
 - You'll write an entry for 1 of each: activation function, loss function, regularizer, and optimizer.
 - Completion grade
 - Deadline is last day of class (Wednesday, 12/10)
- **No class on the Wednesday before Thanksgiving** (Wednesday, 11/26)
- **First week of December** (12/1 and 12/3): will plan to schedule one-on-one project consultations
- **Second week of December** (12/8 and 12/10): Final presentations (~4-5 people per class session)
 - 10 minute talks + 5 min Q&A

Quick recap

Quick Recap: Dimensionality Reduction

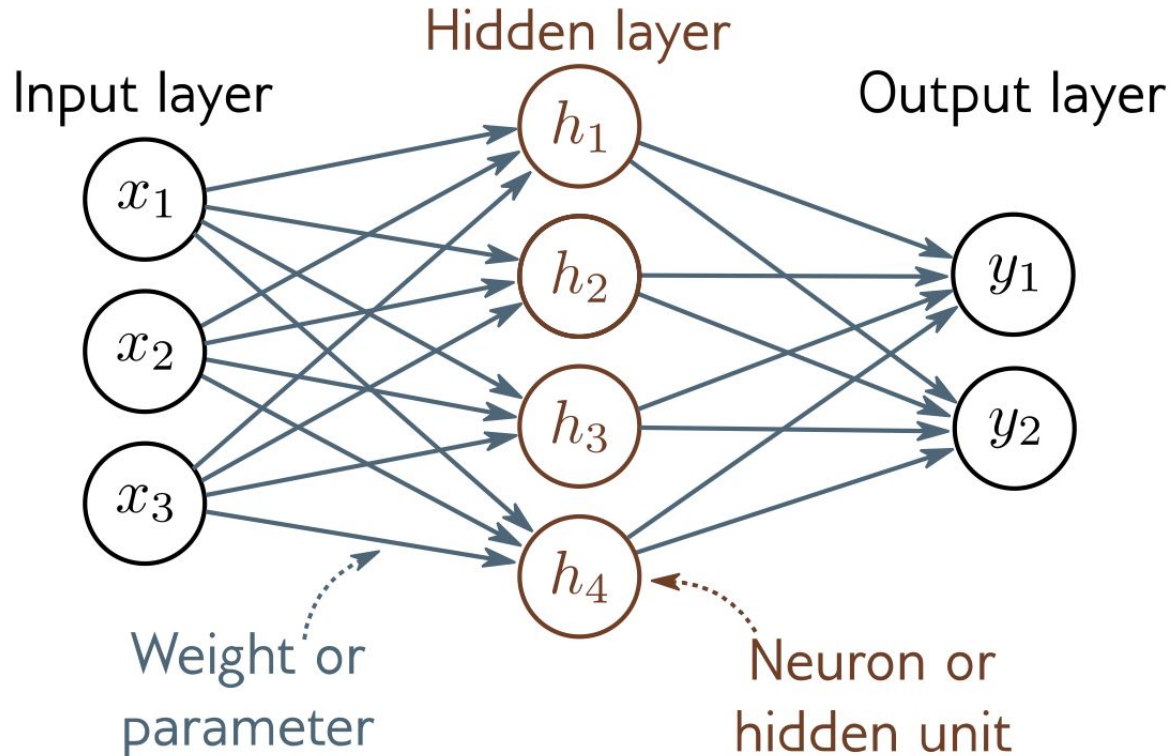
- Our data might have millions of dimensions, but the true underlying physics phenomena we care about live on a lower-dimensional manifold.
- Dimensionality-reduction techniques can help us **project our complex experimental data into lower-dimensional spaces**.
 - Nice for visualization, clustering, and noise reduction
- **Linear method:** PCA
- **Non-linear methods:** t-SNE, UMAP, Autoencoders (AEs)



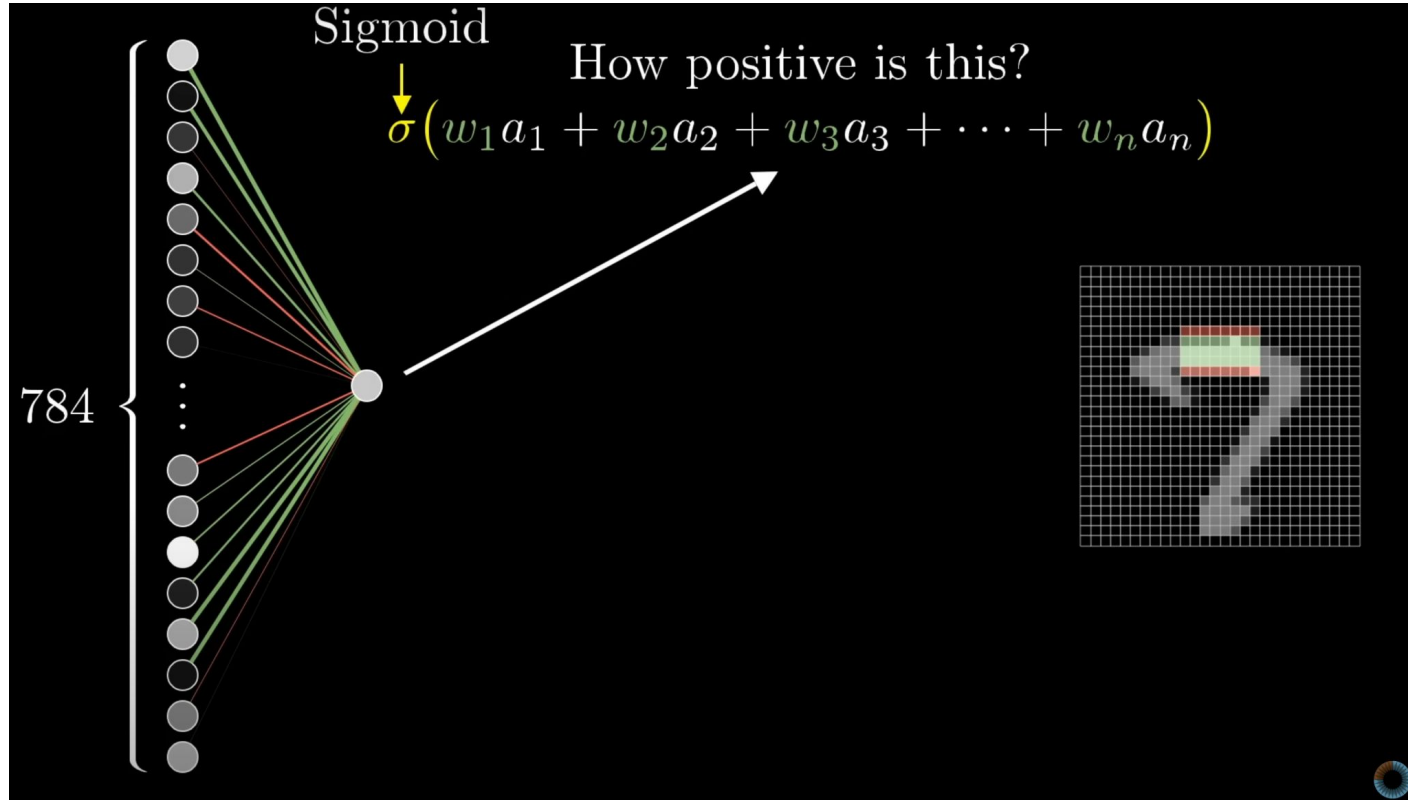
Activations & optimizers

Activation functions

Activation functions add meaning to our weighted sum



Activation functions add meaning to our weighted sum



Activation functions introduce nonlinearities

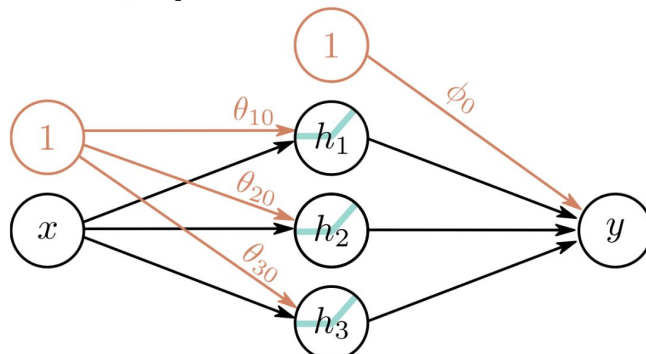
Let's consider a shallow neural network that converts inputs x into predictions y :

$$h_1 = a[\theta_{10} + \theta_{11}x]$$

$$h_2 = a[\theta_{20} + \theta_{21}x]$$

$$h_3 = a[\theta_{30} + \theta_{31}x]$$

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$



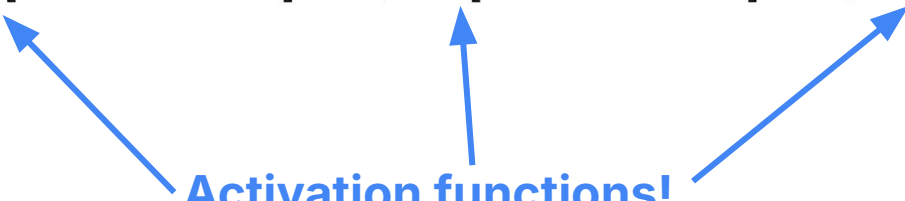
Each parameter multiplies its source and adds to its target

Activation functions introduce nonlinearities

Let's consider a shallow neural network that converts inputs x into predictions y :

$$\begin{aligned} y &= f[x, \phi] \\ &= \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x] \end{aligned}$$

Activation functions!



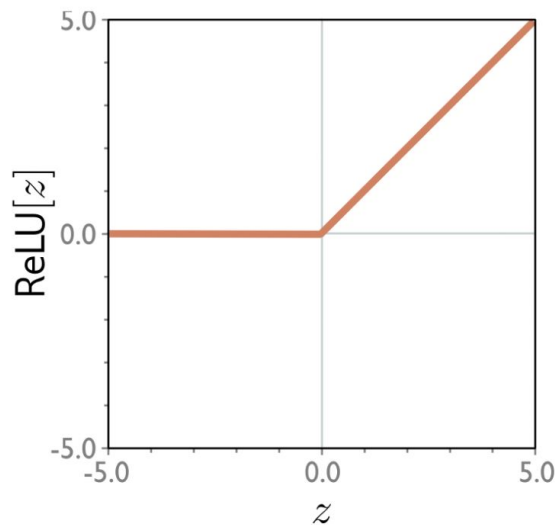
Activation functions introduce nonlinearities

We'll pick the ReLU activation:

$$a[z] = \text{ReLU}[z] = \begin{cases} 0 & z < 0 \\ z & z \geq 0 \end{cases}.$$

Rectified Linear Unit

(particular kind of activation function)

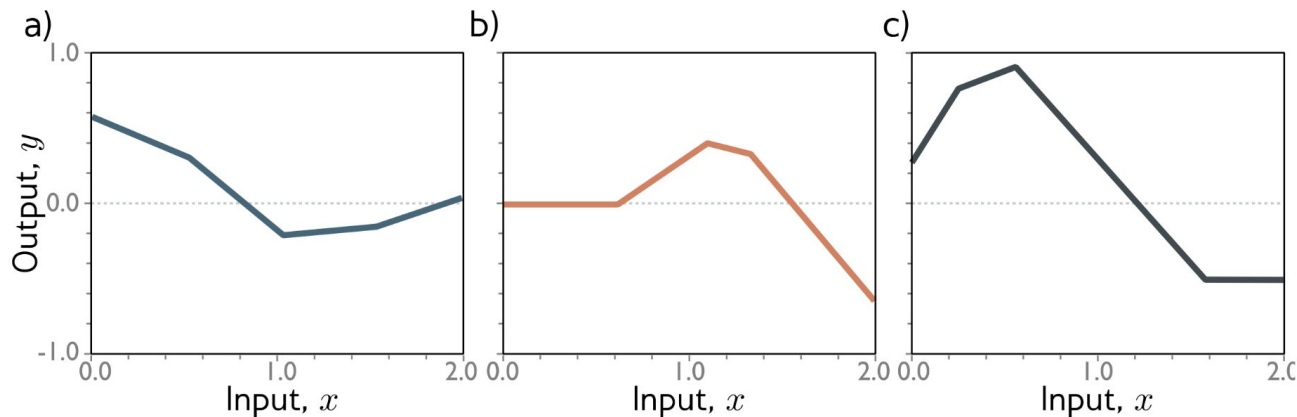


Activation functions introduce nonlinearities

For various choices of NN parameters, we can start to see the nonlinearities emerge:

of activation functions = # of nonlinearities in your approximated function!

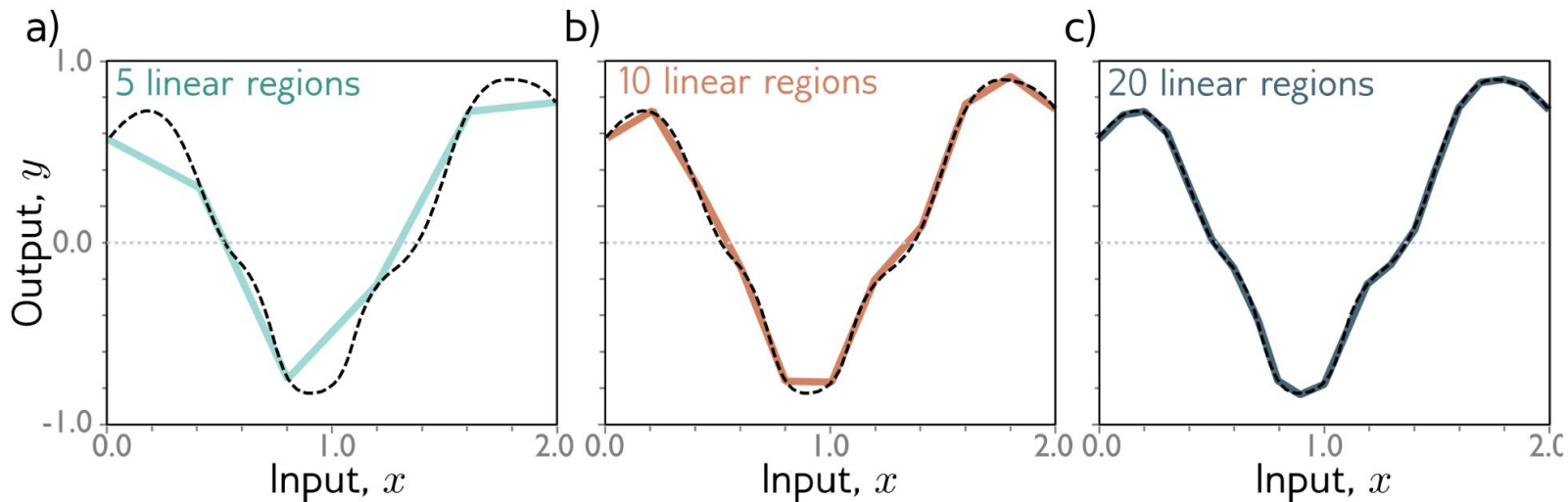
$$y = \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x].$$



Piecewise linear functions with three joints

Activation functions introduce nonlinearities

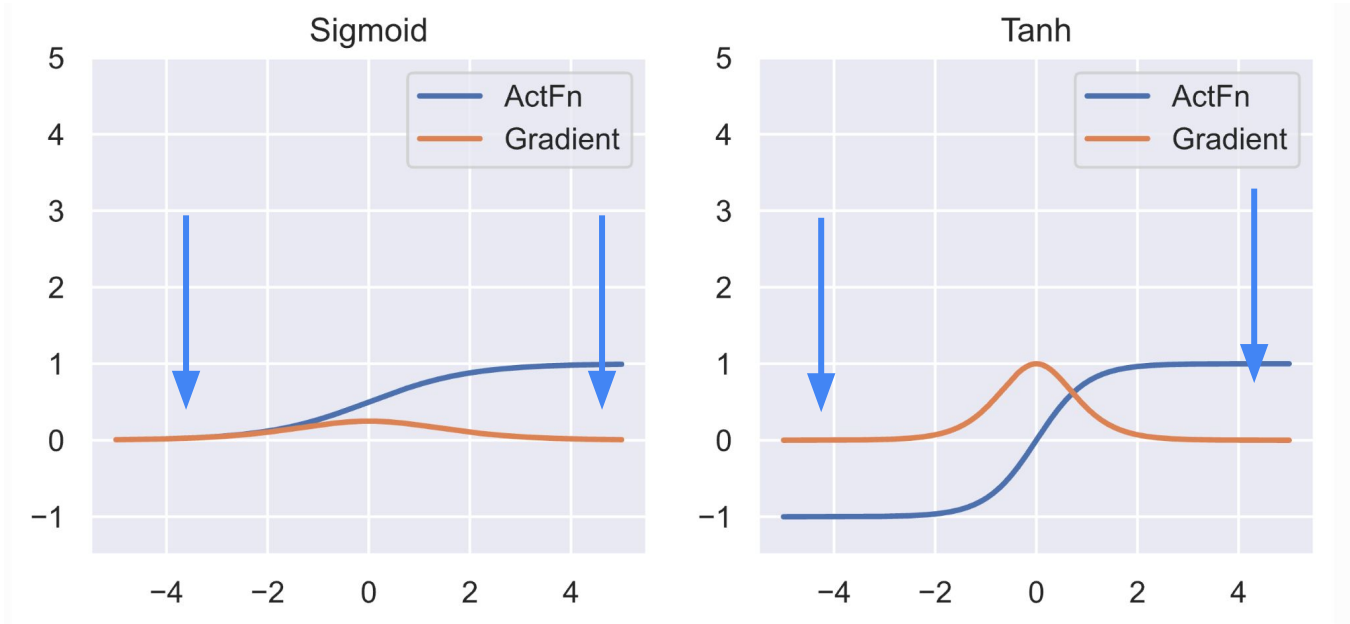
As we add more nonlinearities (i.e. activation functions), we can start to see why the universal approximation theorem holds:



The vanishing gradient problem

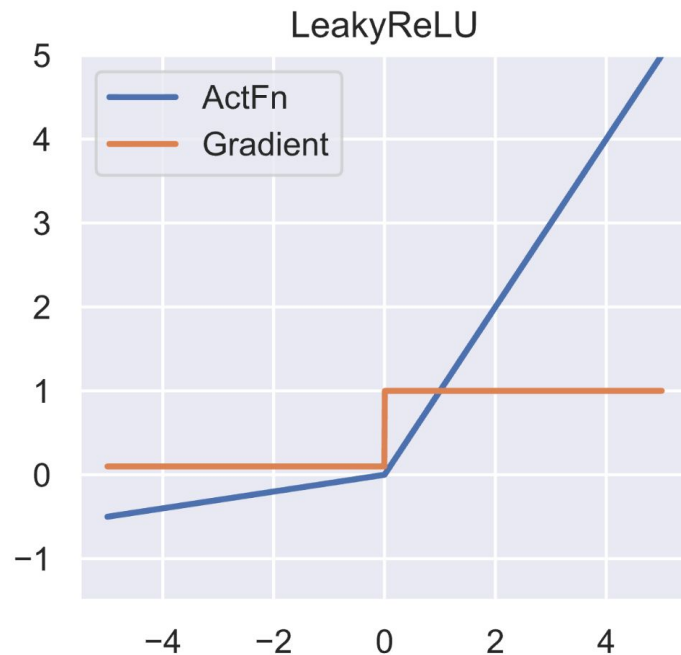
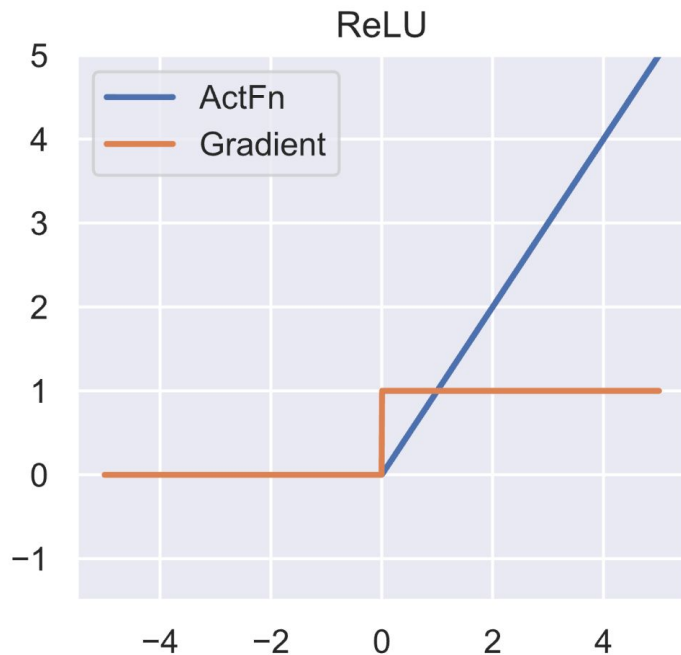
In “early” ML history, the most common activation functions were **sigmoid** & **tanh**.

However, deep NNs using these activations struggled, as derivatives for more extreme values were ~ 0 , and this effect compounded as more and more layers were added:



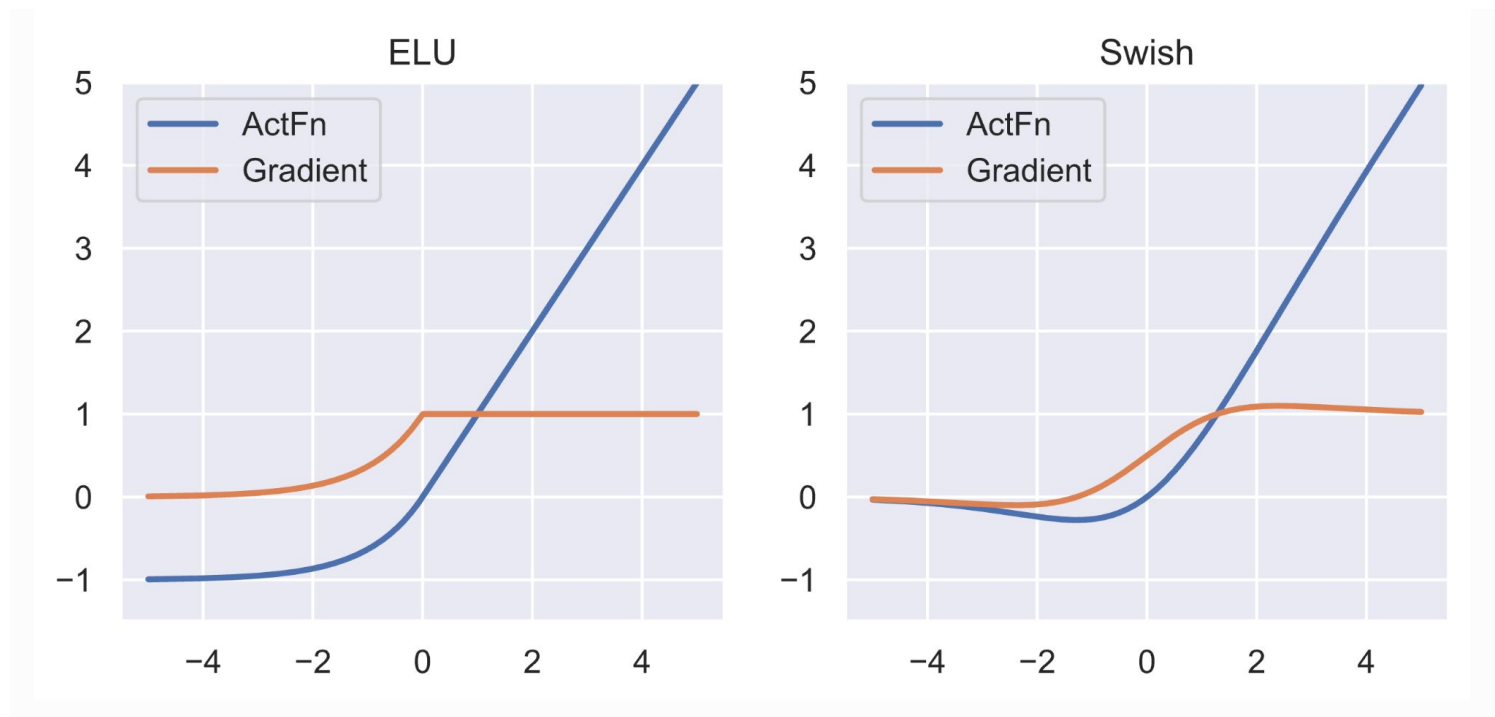
Solution: ReLU, Leaky ReLU, and more

The re-popularization of ReLU circa ~2010 helped deep neural networks finally take off:



Solution: ReLU, Leaky ReLU, and more

Other smoother gradients have also been proposed – these can help in some cases:



Some possibly helpful rules of thumb

- ReLU or Leaky ReLU are good default choices for hidden layer activations
- Sigmoid / tanh should *not* be used for hidden layer activations
- Swish, ELU, etc. are particularly helpful if you have ~40+ hidden layers
- Your final activation function depends on the task at hand:
 - Binary Classification? → Sigmoid
 - Multiclass Classification? → Softmax
 - Regression? → No activation

Optimizers

Optimizers define the model's learning *strategy*.



Most thorough: Gradient Descent

Step 1. Compute the derivatives of the loss with respect to the parameters:

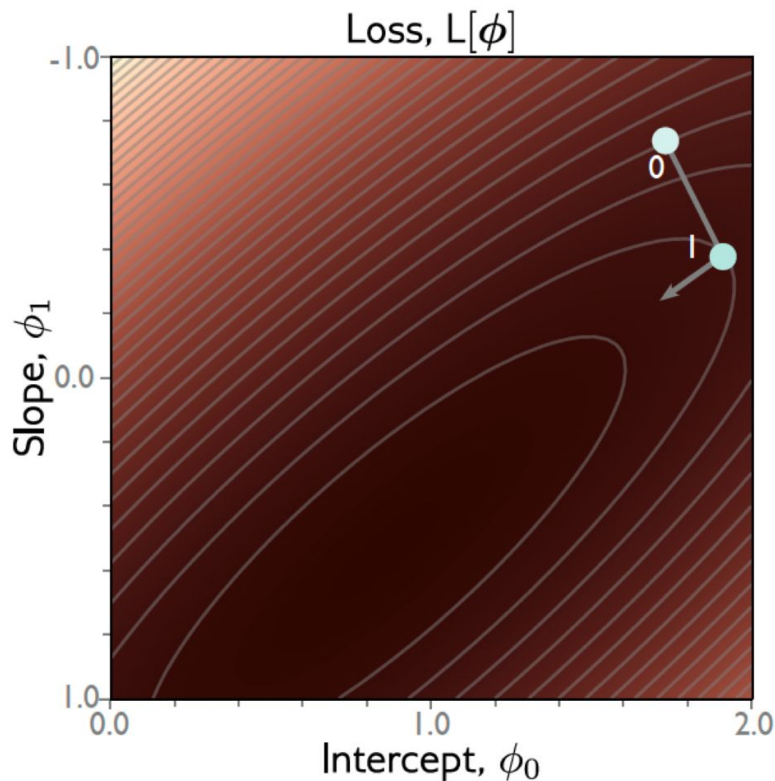
$$\frac{\partial L}{\partial \phi} = \begin{bmatrix} \frac{\partial L}{\partial \phi_0} \\ \frac{\partial L}{\partial \phi_1} \\ \vdots \\ \frac{\partial L}{\partial \phi_N} \end{bmatrix}.$$

Step 2. Update the parameters according to the rule:

$$\phi \longleftarrow \phi - \alpha \frac{\partial L}{\partial \phi},$$

where the positive scalar α determines the magnitude of the change.

Most thorough: Gradient Descent



Step 1: Compute derivatives (slopes of function) with Respect to the parameters

$$\frac{\partial L}{\partial \phi} = \frac{\partial}{\partial \phi} \sum_{i=1}^I \ell_i = \sum_{i=1}^I \frac{\partial \ell_i}{\partial \phi}$$

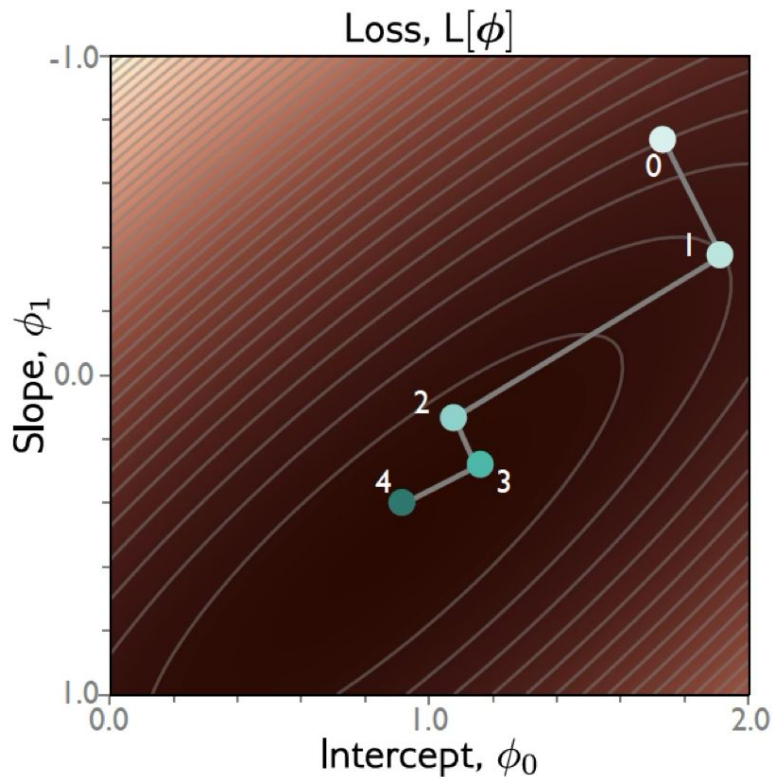
$$\frac{\partial \ell_i}{\partial \phi} = \begin{bmatrix} \frac{\partial \ell_i}{\partial \phi_0} \\ \frac{\partial \ell_i}{\partial \phi_1} \end{bmatrix} = \begin{bmatrix} 2(\phi_0 + \phi_1 x_i - y_i) \\ 2x_i(\phi_0 + \phi_1 x_i - y_i) \end{bmatrix}$$

Step 2: Update parameters according to rule

$$\phi \leftarrow \phi - \alpha \frac{\partial L}{\partial \phi}$$

α = step size

Most thorough: Gradient Descent



Step 1: Compute derivatives (slopes of function) with Respect to the parameters

$$\frac{\partial L}{\partial \phi} = \frac{\partial}{\partial \phi} \sum_{i=1}^I \ell_i = \sum_{i=1}^I \frac{\partial \ell_i}{\partial \phi}$$

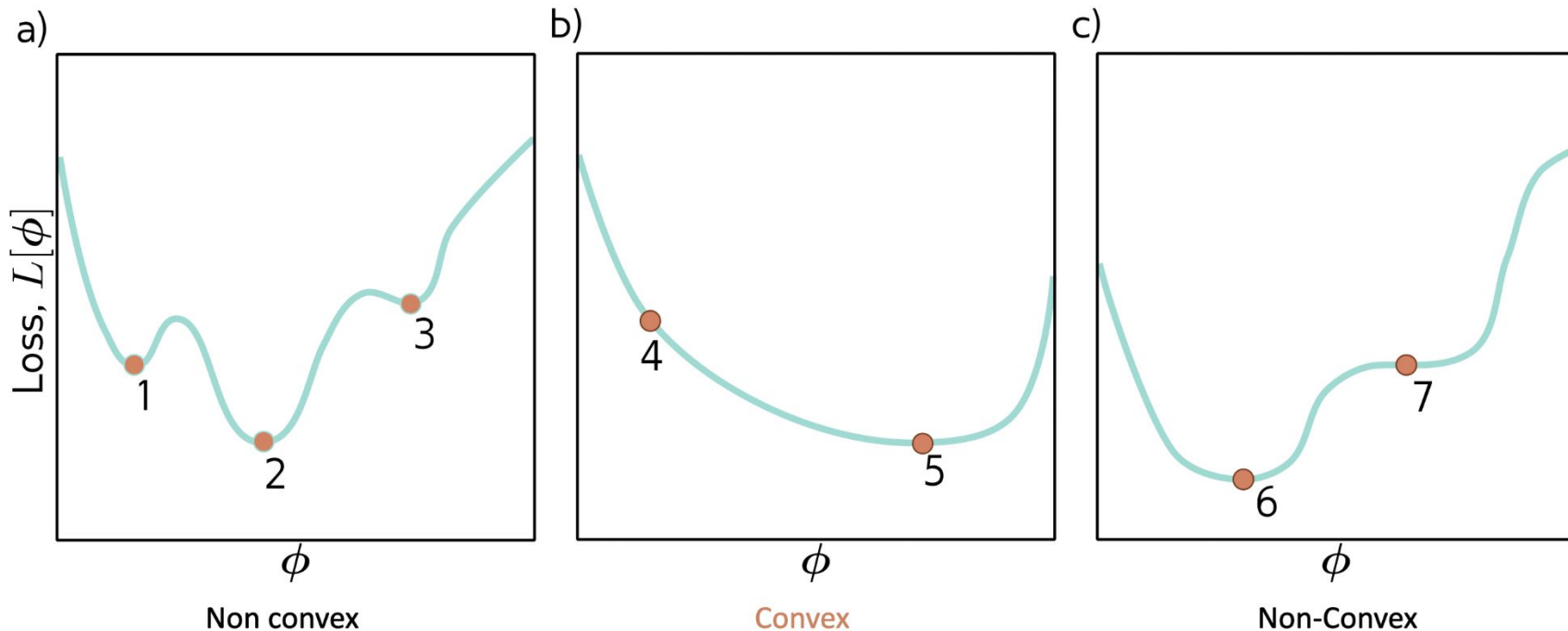
$$\frac{\partial \ell_i}{\partial \phi} = \begin{bmatrix} \frac{\partial \ell_i}{\partial \phi_0} \\ \frac{\partial \ell_i}{\partial \phi_1} \end{bmatrix} = \begin{bmatrix} 2(\phi_0 + \phi_1 x_i - y_i) \\ 2x_i(\phi_0 + \phi_1 x_i - y_i) \end{bmatrix}$$

Step 2: Update parameters according to rule

$$\phi \longleftarrow \phi - \alpha \frac{\partial L}{\partial \phi}$$

α = step size

More efficient & nimble: Stochastic Gradient Descent (SGD)



Test for convexity is that 2nd derivative is positive everywhere

More efficient & nimble: Stochastic Gradient Descent (SGD)

- Train on mini-batches instead of full dataset
- This has some nice consequences:
 - Adds noise / stochasticity
 - Can potentially go uphill at times
 - More computationally efficient
 - Less chance of getting stuck

Before (full batch descent)

$$\phi_{t+1} \longleftarrow \phi_t - \alpha \sum_{i=1}^I \frac{\partial \ell_i[\phi_t]}{\partial \phi},$$

After (SGD)

$$\phi_{t+1} \longleftarrow \phi_t - \alpha \sum_{i \in \mathcal{B}_t} \frac{\partial \ell_i[\phi_t]}{\partial \phi},$$

Fixed learning rate α

More additions: Momentum & Adaptive Moment Estimation

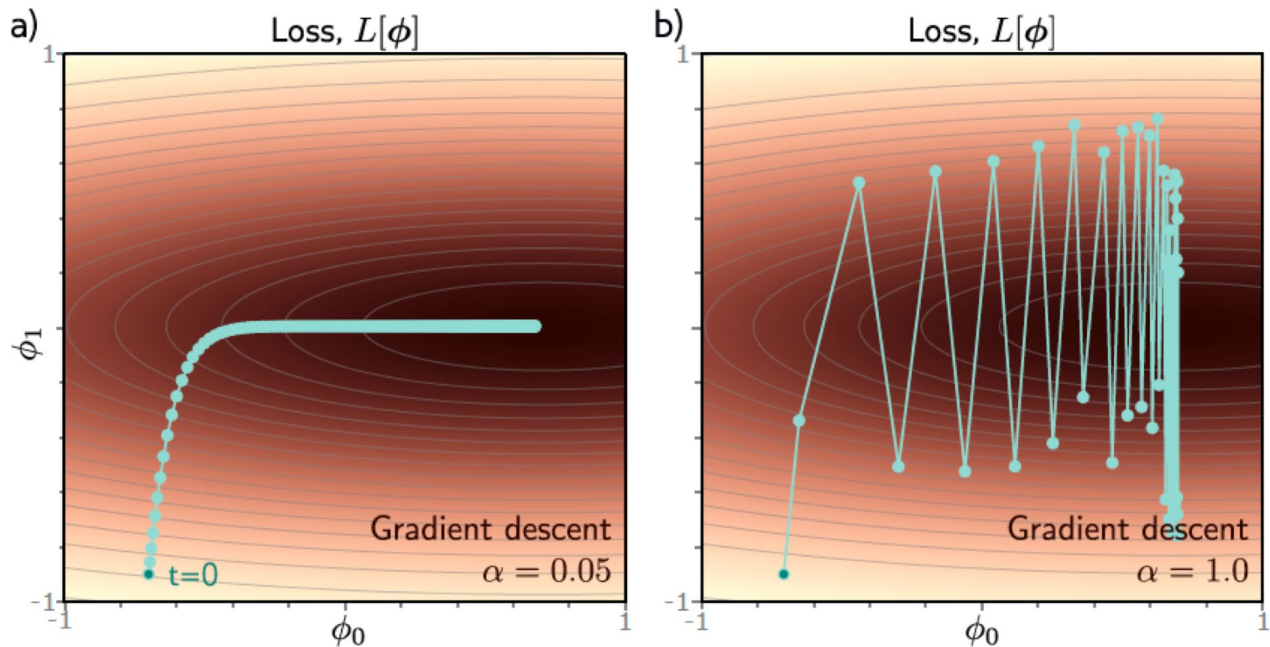
- **Momentum** can help smooth the gradient descent by retaining some “memory” of the previous steps (i.e. a weighted average over the history)
 - Weighted sum of this gradient and previous gradient

$$\mathbf{m}_{t+1} \leftarrow \beta \cdot \mathbf{m}_t + (1 - \beta) \sum_{i \in \mathcal{B}_t} \frac{\partial \ell_i[\phi_t]}{\partial \phi}$$

$$\phi_{t+1} \leftarrow \phi_t - \alpha \cdot \mathbf{m}_{t+1}$$

More additions: Momentum & Adaptive Moment Estimation

- **Adam**, i.e. "adaptive moment estimation", adapts the step sizes per parameter



More additions: Momentum & Adaptive Moment Estimation

- **Adam**, i.e. "adaptive moment estimation", adapts the step sizes per parameter
 - Measure mean and pointwise squared gradient

$$\mathbf{m}_{t+1} \leftarrow \frac{\partial L[\phi_t]}{\partial \phi}$$
$$\mathbf{v}_{t+1} \leftarrow \frac{\partial L[\phi_t]^2}{\partial \phi}$$

- Normalize:

$$\phi_{t+1} \leftarrow \phi_t - \alpha \cdot \frac{\mathbf{m}_{t+1}}{\sqrt{\mathbf{v}_{t+1}} + \epsilon}$$

More additions: Momentum & Adaptive Moment Estimation

- **Adam**, i.e. "adaptive moment estimation", adapts the step sizes per parameter

- Compute mean and pointwise squared gradients with momentum $\mathbf{m}_{t+1} \leftarrow \beta \cdot \mathbf{m}_t + (1 - \beta) \frac{\partial L[\phi_t]}{\partial \phi}$

$$\mathbf{v}_{t+1} \leftarrow \gamma \cdot \mathbf{v}_t + (1 - \gamma) \left(\frac{\partial L[\phi_t]}{\partial \phi} \right)^2$$

- Moderate near start of the sequence

$$\tilde{\mathbf{m}}_{t+1} \leftarrow \frac{\mathbf{m}_{t+1}}{1 - \beta^{t+1}}$$

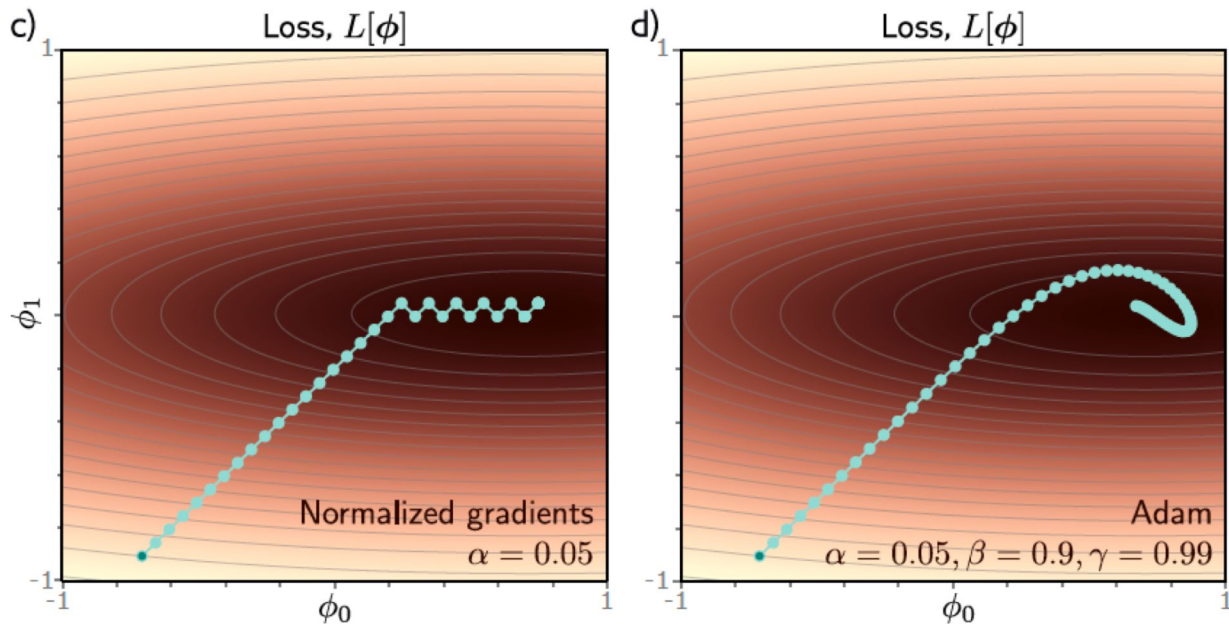
$$\tilde{\mathbf{v}}_{t+1} \leftarrow \frac{\mathbf{v}_{t+1}}{1 - \gamma^{t+1}}$$

- Update the parameters

$$\phi_{t+1} \leftarrow \phi_t - \alpha \cdot \frac{\tilde{\mathbf{m}}_{t+1}}{\sqrt{\tilde{\mathbf{v}}_{t+1} + \epsilon}}$$

More additions: Momentum & Adaptive Moment Estimation

- **Adam**, i.e. "adaptive moment estimation", adapts the step sizes per parameter



In-class work

Activation functions:

https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/tutorial3/Activation_Functions.html (or search: "UvA Tutorial 3: Activation Functions")

Optimizers:

<https://distill.pub/2017/momentum/>